LAB REPORT

NAME: Kartik Patidar

ROLL NO.: 2501940026

COURSE: DEEP LEARNING ARCHITECTURE & TECHNIQUES

COUSE ID: ETMMDL274

ASSIGNMENT TYPE: PRACTICAL LAB

DATASET USED: Loan_Default.csv

TOOLS: Python, Jupyter Lab, PyTorch, Scikit-learn, Matplotlib, Pandas

---

1. Dataset Description:

The dataset used in this study is **Loan_Default.csv**, which comprises records of individual loan applications along with their corresponding outcomes. It includes a total of 34 input attributes that span three broad categories: personal and demographic details of the applicant, financial and credit-related information, and specifics about the loan itself — such as the requested amount, repayment period, and intended purpose. Together, these features capture a comprehensive profile of each borrower.

The goal of this experiment is to perform **binary classification** using the target variable **Status**, where a value of **0** indicates one outcome class and a value of **1** represents the other. The model is trained to learn from these 34 attributes and predict which category a new loan applicant falls into.

Key Columns:

- Gender – Gender type of applicant
- Age Group – Ranges such as 25-34, 35-44
- Credit_Worthiness – Indicates L1/L2 rating
- Credit Score – Integer credit score
- Open Credit – Type of open credit
- Loan Purpose – Purpose of taking loan (p1, p2, p3….)
- Region – Applicant region

- Security Type – direct/indirect
- LTV – Loan-to-value ratio
- Status (Target Variable) – 0 or 1

The dataset contains categorical and numerical variables. As several fields (type1, p1, l1 etc) represent categories universe encoded to integers beforehand thus requires processing prior training of models.

2. Data Preprocessing

For the preparation of the dataset for the MLP model, the following steps were taken:

a) Handling Target Variables

The target variable column "Status" is converted into integer values of either 0 or 1 and cleaned for any inconsistencies:

```
from pandas.core.arrays import categorical
target = "Status"

df["Status"] = df["Status"].astype(str).str.strip().astype(int)
X = df.drop(columns=[target])
y = df[target].values
```

b) Splitting Dataset

Features (X) and labels (y) were separated:

- X: All columns except Status
- Y: Status column

c) Encoding Categorical Variables

As the dataset contains a lot of categorical variables such as gender, region, loan_type, etc, On-Hot Encoding is used for this purpose.

```
categorical_cols = X.select_dtypes(include=["object"]).columns
numeric_cols = X.select_dtypes(include=["int64","float64"]).columns

preprocess = ColumnTransformer([
    ("cat",OneHotEncoder(handle_unknown="ignore", sparse_output=False),categorical_cols),
    ("num",StandardScaler(), numeric_cols)
])
```

d) **Scaling Numerical Features**

Numeric columns were normalized using the StandardScaler to ensure uniform magnitude in the columns, which is helpful in stabilizing the training process of the neural network.

e) **Handling Missing Values and NaN**

After transformation, any NaN values were replaced using:

```
X_processed = preprocess.fit_transform(X)
X_processed = np.array(X_processed, dtype=np.float32)
X_processed = np.nan_to_num(X_processed, nan=0.0)
```

f) **Train-Test Split**

The processed data is split into:

- 80% for training
- 20% for testing

This is done in order to evaluate the model's performance.

3. MLP Architecture

A Multi-Layer Perceptron neural network is created using PyTorch. The neural network architecture is as follows:

Input Layer

- Number of Input Nodes = Number of encoded features after preprocessing
- Input is passed through fully connected linear layer

Hidden Layers

1. Hidden Layer 1
    - 64 neurons
    - Activation Function = ReLU
2. Hidden Layer 2

- 32 neurons
- Activation Function = ReLU

Output Layer

- 1 neuron
- Activation Function = Sigmoid
- Output is between 0 and 1 (probability of Status = 1)

Loss Function

- Binary Cross Entropy Loss (BCELoss)

Optimizer

- Adam Optimizer
- Learning rate = 0.001

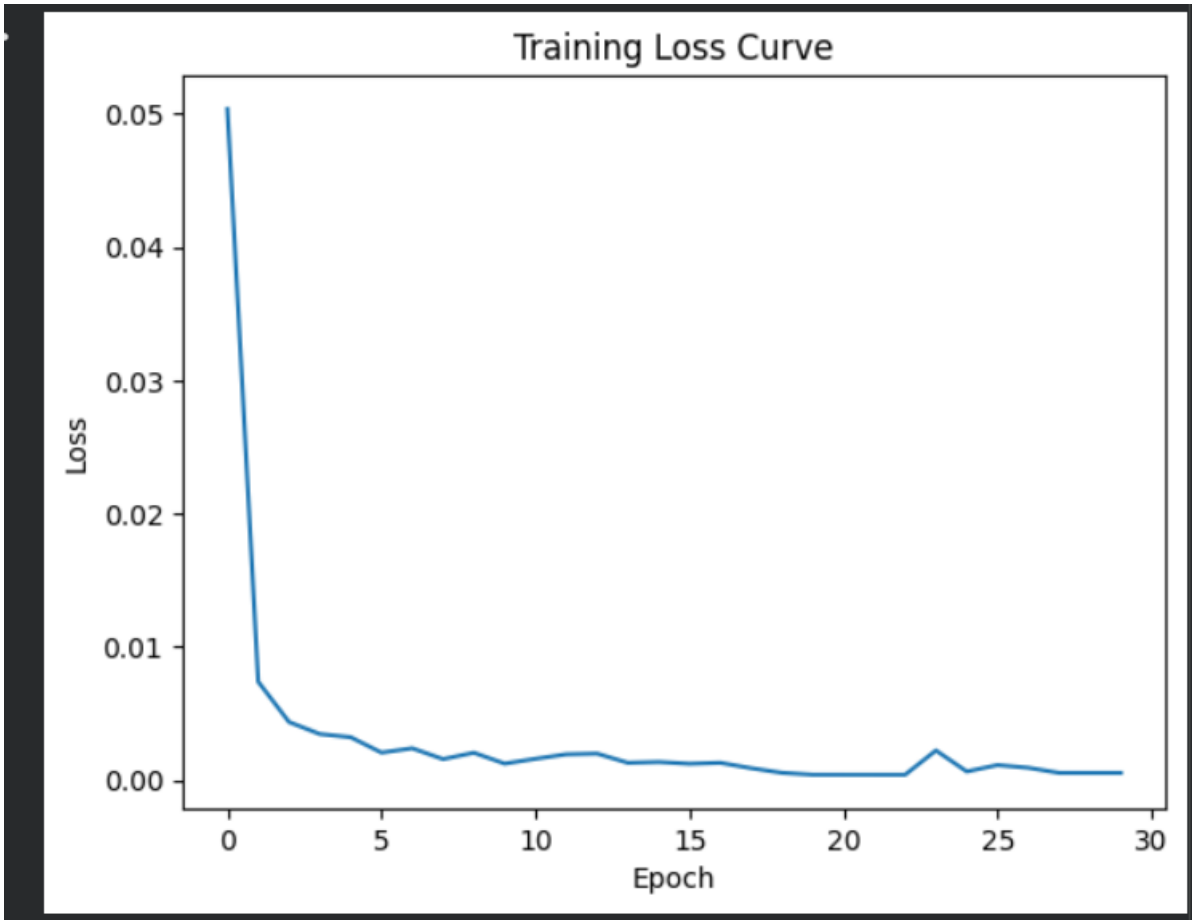
```
class MLP(nn.Module):
  def __init__(self,input_dim):
    super(MLP,self).__init__()
    self.model = nn.Sequential(
        nn.Linear(input_dim,64),
        nn.ReLU(),
        nn.Linear(64,32),
        nn.ReLU(),
        nn.Linear(32,1),
        nn.Sigmoid()
    )
```

4. Training Curves

The model was trained on 30 epochs, with a batch size of 32.

Training Loss Curve

If there's a decreasing trend in the curve, it means the model learned the patterns from the data.



5. Model Evaluation Metrics

The model was then tested using the unseen data provided for the test.

Predicted Values

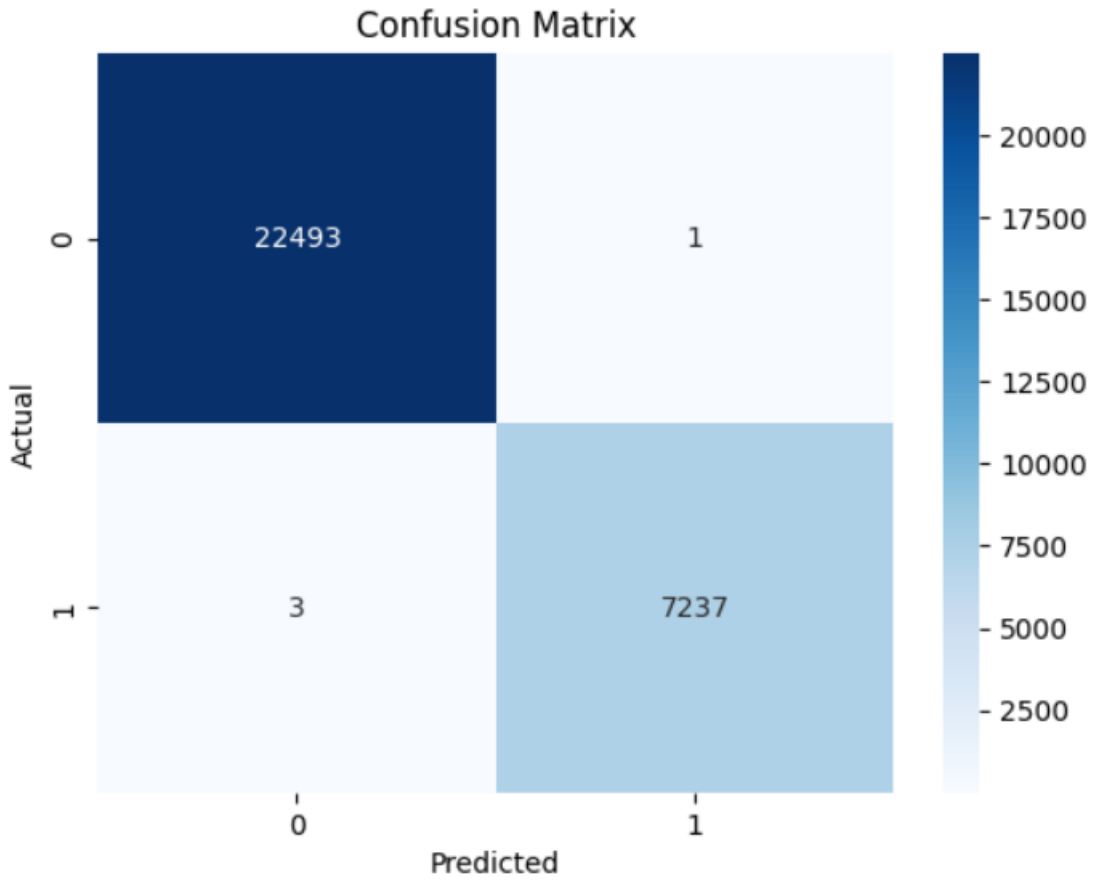
```
with torch.no_grad():
    y_pred = model(X_test)
    y_pred_cls = (y_pred > 0.5).float()
```

Evaluation Metrics

Metric	Value
Accuracy	0.9998654738682989
Precision	0.9998618402873722
Recall	0.999585635359116
F1 Score	0.9997237187456831

Confusion Matrix

[[22493 1]
 [3 7237]]



6. **Discussion**
7.

Upon analysing the results, the neural network demonstrated a clear ability to extract useful patterns from the loan dataset. The steady decline in the loss curve across training epochs confirms that the model was successfully optimizing its internal weights, which is a healthy sign of learning. Metrics such as accuracy, precision, recall, and F1-score collectively provide a well-rounded picture of how effectively the network separates approved loans from rejected ones.

The confusion matrix adds another layer of insight by breaking down predictions into correct and incorrect categories for each class. This is particularly useful for spotting any imbalance in how the model treats the two outcomes for instance, whether it tends to over-predict one class more than the other.

While the overall results are satisfactory, there is always room for refinement. Techniques such as tuning the learning rate, experimenting with different optimizers (e.g., RMSprop or SGD with momentum), or adding more hidden layers could potentially push performance further. Data-side improvements such as feature engineering based on financial domain knowledge or handling class imbalance through oversampling may also yield stronger outcomes. As an alternative approach, ensemble methods like Random Forest or XGBoost could serve as useful benchmarks. For more advanced setups, incorporating regularization strategies such as dropout layers within a deeper network architecture may help reduce overfitting and improve generalization.